

Frontend Deployment Guide - Kidrove.com

Current Configuration

- **Domain:** kidrove.com
- **Server:** Hostinger VPS (93.127.185.245)
- **Backend:** http://93.127.185.245:5000/api
- **Framework:** React + Vite

⚠ CRITICAL: Fix API URL First

Your frontend is configured with HTTP backend URL, but production should use HTTPS:

Update frontend/.env:

```
# Change from:
VITE_API_BASE_URL=http://93.127.185.245:5000/api

# To (using domain):
VITE_API_BASE_URL=https://api.kidrove.com/api
```

OR if using IP with HTTPS:

```
VITE_API_BASE_URL=https://93.127.185.245:5001/api
```

Deployment Steps

Step 1: Build Frontend Locally

On your local machine (Windows):

```
cd E:\coding\gema\frontend

# Install dependencies (if not already done)
npm install

# Build for production
npm run build
```

This creates a `dist` folder with optimized static files.

Step 2: Upload to Server

Option A: Using Git (Recommended)

```
# Commit your changes
git add .
git commit -m "Frontend production build ready"
git push origin backend_auth

# SSH into server
ssh root@93.127.185.245

# Navigate to project
cd /var/www/GEMA-Project/frontend

# Pull latest changes
git pull origin backend_auth

# Install dependencies
npm install

# Build on server
npm run build
```

Option B: Using SCP/SFTP

```
# From local machine
scp -r E:\coding\gema\frontend\dist root@93.127.185.245:/var/www/GEMA-Project/frontend/
```

Step 3: Install and Configure Nginx

SSH into your server and run:

```
# Install nginx
sudo apt update
sudo apt install -y nginx

# Check nginx status
sudo systemctl status nginx
```

Step 4: Create Nginx Configuration

```
sudo nano /etc/nginx/sites-available/kidrove
```

Paste this configuration:

```
# Kidrove Frontend Configuration

# Redirect HTTP to HTTPS
server {
    listen 80;
    listen [::]:80;
    server_name kidrove.com www.kidrove.com;

    # Redirect to HTTPS
    return 301 https://$server_name$request_uri;
}

# HTTPS Server Block
server {
    listen 443 ssl http2;
    listen [::]:443 ssl http2;
    server_name kidrove.com www.kidrove.com;

    # SSL Configuration (Let's Encrypt certificates)
    ssl_certificate /etc/letsencrypt/live/kidrove.com/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/kidrove.com/privkey.pem;
    ssl_protocols TLSv1.2 TLSv1.3;
    ssl_ciphers HIGH:!aNULL:!MD5;
    ssl_prefer_server_ciphers on;

    # Root directory
    root /var/www/GEMA-Project/frontend/dist;
    index index.html;

    # Gzip compression
    gzip on;
    gzip_vary on;
    gzip_min_length 1024;
    gzip_types text/plain text/css text/xml text/javascript application/javascript application/json
application/xml+rss application/rss+xml font/truetype font/opentype application/vnd.ms-fontobject
image/svg+xml;

    # Security headers
    add_header X-Frame-Options "SAMEORIGIN" always;
    add_header X-Content-Type-Options "nosniff" always;
    add_header X-XSS-Protection "1; mode=block" always;
    add_header Referrer-Policy "no-referrer-when-downgrade" always;
    add_header Content-Security-Policy "default-src 'self' https: data: 'unsafe-inline' 'unsafe-eval';
img-src 'self' https: data: blob:; font-src 'self' https: data:;" always;

    # React Router - try files first, then fallback to index.html
    location / {
        try_files $uri $uri/ /index.html;
```

```

}

# Cache static assets
location ~* \.(js|css|png|jpg|jpeg|gif|ico|svg|woff|woff2|ttf|eot)$ {
    expires 1y;
    add_header Cache-Control "public, immutable";
}

# Service worker should not be cached
location = /sw.js {
    add_header Cache-Control "no-cache";
    proxy_cache_bypass $http_pragma;
}

# API Proxy (optional - if you want to proxy backend through nginx)
location /api/ {
    proxy_pass http://localhost:5000/api/;
    proxy_http_version 1.1;
    proxy_set_header Upgrade $http_upgrade;
    proxy_set_header Connection 'upgrade';
    proxy_set_header Host $host;
    proxy_cache_bypass $http_upgrade;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;
}

# Error pages
error_page 404 /index.html;
}

```

Step 5: Install SSL Certificate (Let's Encrypt)

```

# Install Certbot
sudo apt install -y certbot python3-certbot-nginx

# Obtain SSL certificate
sudo certbot --nginx -d kidrove.com -d www.kidrove.com

# Follow prompts:
# - Enter your email
# - Agree to terms
# - Choose to redirect HTTP to HTTPS (option 2)

# Test auto-renewal
sudo certbot renew --dry-run

```

Step 6: Enable and Test Nginx

```

# Enable the site
sudo ln -s /etc/nginx/sites-available/kidrove /etc/nginx/sites-enabled/

# Remove default site (optional)
sudo rm /etc/nginx/sites-enabled/default

# Test nginx configuration
sudo nginx -t

# If test passes, reload nginx
sudo systemctl reload nginx

# Enable nginx to start on boot
sudo systemctl enable nginx

```

Step 7: Configure Backend for HTTPS (IMPORTANT!)

Option A: Use Nginx as Reverse Proxy (Recommended)

If you used the nginx config above with `/api/` proxy, update your backend CORS:

```
nano /var/www/GEMA-Project/backend/.env
```

Add/update:

```
FRONTEND_URL=https://kidrove.com
ADDITIONAL_ALLOWED_ORIGINS=https://www.kidrove.com
```

Option B: Direct HTTPS to Backend

Create SSL certificate for API subdomain:

```
# Point api.kidrove.com to your server IP in DNS
# Then:
sudo certbot --nginx -d api.kidrove.com

# Update backend to use HTTPS (requires code changes)
# Or use nginx reverse proxy (easier)
```

Step 8: Update DNS Records

In your domain registrar (Hostinger DNS):

Type	Name	Value	TTL
A	@	93.127.185.245	3600
A	www	93.127.185.245	3600
A	api	93.127.185.245	3600 (if using api subdomain)

Step 9: Test Deployment

```
# Check nginx is running
sudo systemctl status nginx

# Check nginx error logs if issues
sudo tail -f /var/log/nginx/error.log

# Test SSL certificate
curl -I https://kidrove.com

# Test from browser
open https://kidrove.com
```

Quick Deploy Script

Create /var/www/GEMA-Project/deploy-frontend.sh:

```
#!/bin/bash

echo "🚀 Deploying Kidrove Frontend..."

cd /var/www/GEMA-Project/frontend

# Pull latest code
echo "📦 Pulling latest code..."
git pull origin backend_auth

# Install dependencies
echo "📦 Installing dependencies..."
npm install

# Build
echo "🔨 Building frontend..."
npm run build

# Reload nginx
echo "🔄 Reloading nginx..."
sudo systemctl reload nginx

echo "✅ Frontend deployed successfully!"
echo "🌐 Visit: https://kidrove.com"
```

Make executable and run:

```
chmod +x /var/www/GEMA-Project/deploy-frontend.sh
/var/www/GEMA-Project/deploy-frontend.sh
```

Troubleshooting

Issue: White screen / Blank page

Solution:

- Check browser console for errors
- Verify API URL in .env is correct
- Check nginx error logs: `sudo tail -f /var/log/nginx/error.log`

Issue: API calls failing (CORS)

Solution:

- Update backend CORS to allow `https://kidrove.com`
- Check backend .env has correct `FRONTEND_URL`
- Restart backend: `pm2 restart gema-backend`

Issue: SSL certificate errors

Solution:

```
# Renew certificate
sudo certbot renew

# Check certificate status
sudo certbot certificates

# Force renewal
sudo certbot renew --force-renewal
```

Issue: 502 Bad Gateway

Solution:

- Backend not running: `pm2 status`
- Check backend logs: `pm2 logs gema-backend`
- Restart backend: `pm2 restart gema-backend`

Issue: Files not updating

Solution:

```
# Clear browser cache (Ctrl+Shift+Delete)
# Or rebuild and clear nginx cache
cd /var/www/GEMA-Project/frontend
npm run build
sudo systemctl reload nginx
```

Performance Optimization

Enable HTTP/2

Already enabled in nginx config above (`http2` flag)

Enable Brotli Compression (Advanced)

```
sudo apt install -y nginx-module-brotli
# Configure in nginx (search for brotli nginx configuration)
```

CDN (Optional)

Consider using Cloudflare for:

- Additional DDoS protection
- Global CDN
- Automatic SSL
- Caching

Monitoring

Check Nginx Access Logs

```
sudo tail -f /var/log/nginx/access.log
```

Check Frontend Load Time

```
curl -w "%@" -o /dev/null -s https://kidrove.com <<'EOF'
time_namelookup:  %{time_namelookup}\n
time_connect:    %{time_connect}\n
time_starttransfer:  %{time_starttransfer}\n
time_total:      %{time_total}\n
EOF
```

Security Checklist

- ☒ SSL/TLS enabled (HTTPS)
- ☒ HTTP redirects to HTTPS
- ☒ Security headers configured
- ☒ Gzip compression enabled
- ☒ CORS properly configured
- ☒ API keys not exposed in frontend code (env variables)
- ☒ Firewall configured (ufw)

Summary

1. **Build frontend:** `npm run build`
2. **Upload to server:** Git or SCP
3. **Install nginx:** `sudo apt install nginx`
4. **Configure nginx:** Create `/etc/nginx/sites-available/kidrove`
5. **Get SSL cert:** `sudo certbot --nginx -d kidrove.com`
6. **Enable site:** `sudo ln -s /etc/nginx/sites-available/kidrove /etc/nginx/sites-enabled/`
7. **Test & reload:** `sudo nginx -t && sudo systemctl reload nginx`
8. **Visit:** `https://kidrove.com`

Need help? Check logs:

- **Nginx:** `sudo tail -f /var/log/nginx/error.log`
- **Backend:** `pm2 logs gema-backend`